LAPORAN IOT & JARINGAN SENSOR

Implementasi MQTT pada ESP32 untuk Monitoring Suhu dan Kendali Lampu Empat Ruangan

Disusun oleh:

NAMA	NRP	RUANGAN
Muh. Adzril Aziz	2124600095	Ruangan 1
Rassya Aditia	2124600097	Ruangan 2
Bima Syahputra	2124600115	Ruangan 3
Ferrisca Maylanazwa S	2124600117	Ruangan 4

Tugas ini bertujuan mengimplementasikan komunikasi MQTT menggunakan ESP32 untuk memantau suhu simulasi, mengendalikan lampu, serta memantau status koneksi empat ruangan melalui satu broker Mosquitto.

A. Tabel Konfigurasi PIN tiap Ruangan

1. ESP32 Ruangan 1

Komponen	GPIO	Fungsi
Aht10	SDA (GPIO21), SCL (GPIO 22)	Input analog simulasi suhu
LED lampu 1	GPIO2	Kendali lampu 1 melalui resistor 220 Ω
LED lampu 2	GPIO4	Kendali lampu 2 melalui resistor 220 Ω

2. ESP32 Ruangan 2

Komponen	GPIO	Fungsi
Potensiometer 10K	GPIO5	Input analog simulasi suhu dan kelembapan
LED lampu 1	GPIO2	Kendali lampu 1 melalui resistor 220 Ω
LED lampu 2	GPIO4	Kendali lampu 2 melalui resistor 220 Ω

3. ESP32 Ruangan 3

Komponen	GPIO	Fungsi
Potensiometer 10K	GPIO5	Input analog simulasi suhu dan kelembapan

LED lampu 1	GPIO2	Kendali lampu 1 melalui resistor 220 Ω
LED lampu 2	GPIO4	Kendali lampu 2 melalui resistor 220 Ω

4. ESP32 Ruangan 4

Komponen	GPIO	Fungsi
Potensiometer 10K	GPIO5	Input analog simulasi suhu dan kelembapan
LED lampu 1	GPIO2	Kendali lampu 1 melalui resistor 220 Ω
LED lampu 2	GPIO4	Kendali lampu 2 melalui resistor 220 Ω

B. Konversi Suhu dan Kelembapan Simulasi

Dilakukan konversi suhu untuk simulasi dengan rumus berikut

$$\text{baseTemp} = 20{,}0 + \left(\frac{\text{potValue}}{4095{,}0}\right) \times (45{,}0 - 20{,}0)$$

$$\text{finalTemp} = \text{baseTemp} + \text{tempNoise}$$

(dimana tempNoise adalah nilai acak antara $-0{,}4$ sampai $+0{,}4$)

Kemudian konversi kelembapan sebagai berikut

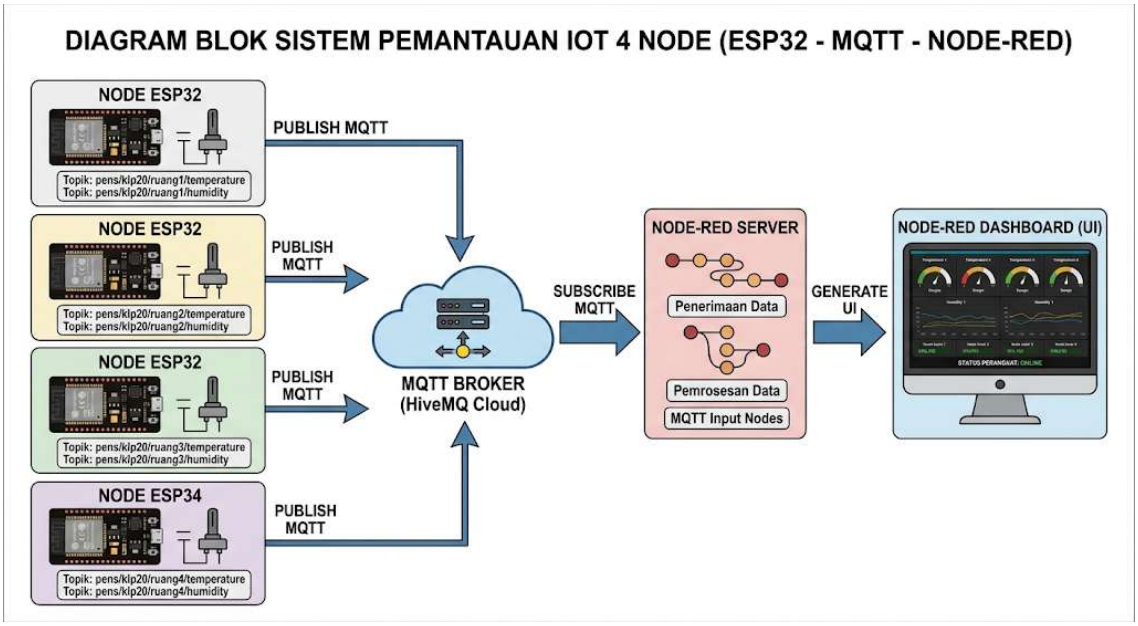
$$\text{baseHum} = 85{,}0 - \left(\frac{\text{finalTemp} - 20{,}0}{45{,}0 - 20{,}0}\right) \times (85{,}0 - 30{,}0)$$

$$\text{finalHum} = \text{clamp}(\text{baseHum} + \text{humNoise}, 10{,}0, 99{,}0)$$

(di mana nilai akhirnya dibatasi di antara $10{,}0\%$ dan $99{,}0\%$)

Pembacaan ADC 12-bit berrentang 0–4095 dikonversi secara matematis menjadi data simulasi suhu ($20,0°\text{C}$ sampai $45,0°\text{C}$) dan kelembapan (berkorelasi terbalik antara $30{,}0\%$ hingga $85{,}0\%$) dengan tambahan variasi acak (*noise*). Nilai kedua parameter tersebut kemudian dikirimkan dalam format dua angka di belakang koma.

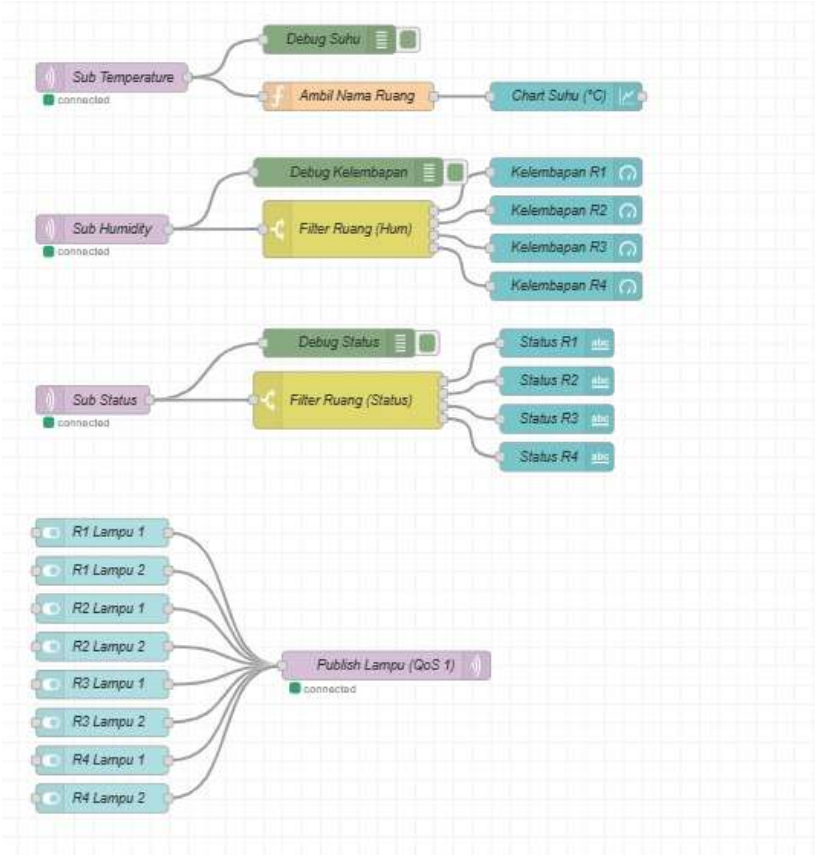
C. Blok Diagram



Gambar 5 Blok Diagram

D. Tabel Topic yang digunakan

Topic	Arah Komunikasi	Contoh Payload	QoS	Retain
pens/klp20/ruangN/temperature	ESP32 → Node-RED	28.5 (°C, 1 angka desimal)	0	Tidak (false)
pens/klp20/ruangN/humidity	ESP32 → Node-RED	65.0 (%, 1 angka desimal)	0	Tidak (false)
pens/klp20/ruangN/lamp1	Node-RED → ESP32	1 (Nyala) atau 0 (Mati)	1	Tidak (false)
pens/klp20/ruangN/lamp2	Node-RED → ESP32	1 (Nyala) atau 0 (Mati)	1	Tidak (false)
pens/klp20/ruangN/status	ESP32 / LWT → Node-RED	ONLINE atau OFFLINE	0	Ya (true)



Gambar 6 Tampilan Node Red

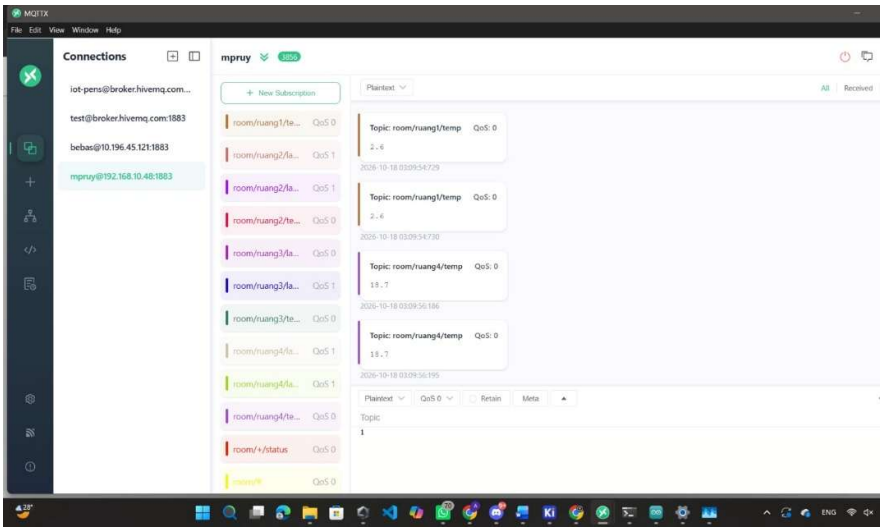
E. Wildcard yang digunakan

No	Topic Wildcard	Fungsi
1	pens/klp20/+/temperature	Memantau data suhu dari seluruh ruangan
2	pens/klp20/+/status	Memantau status online/offline seluruh ruangan
3	pens/klp20/#	Memantau seluruh topic

F. Tabel hasil pengujian Sistem MQTT

No	Kode Uji	Skenario Pengujian	Prosedur & Langkah Uji	Hasil yang Diharapkan
1	T1	Koneksi Wi-Fi Awal	Menyalakan ESP32 Ruang 1 s.d. Ruang 4	ESP32 berhasil tersambung ke jaringan Wi-Fi/Hotspot lokal

2	T2	Koneksi ke Broker MQTT	Memantau Serial Monitor saat ESP32 booting	ESP32 berhasil melakukan koneksi ke Mosquitto Broker (10.32.81.35:1883)
3	T3	Status Online Awal (Retained)	Membuka dashboard Node-RED di browser (/ui)	Indikator status keempat ruangan langsung menampilkan teks online
4	T4	Publish Data Suhu Sensor	Membaca pengiriman data sensor periodik setiap 5 detik	Node-RED menerima data suhu valid dan menampilkannya pada *chart line*
5	T5	Publish Data Kelembapan	Memantau pembaruan nilai kelembapan berkala	Keempat node ui_gauge menampilkan nilai persentase kelembapan dengan akurat



Gambar 7 MQTT Sub

G. Kendala pada pengerjaan

Kendala umum yang sering terjadi pada sistem multi-node ESP32 adalah ketidaksinkronan waktu pengiriman data, di mana data dari beberapa perangkat tidak terkirim secara bersamaan meskipun sama-sama diatur dengan interval 5 detik. Hal ini disebabkan oleh waktu *booting* perangkat yang berbeda-beda serta pergeseran waktu (*time drift*) pada perhitungan millis() di

setiap mikrokontroler, yang membuat siklus pengiriman data antar node saling bergeser dan menumpuk secara acak di broker MQTT.

Sebagai solusi atas permasalahan tersebut, program pada ESP32 dapat disesuaikan dengan memanfaatkan sinkronisasi waktu absolut dari server NTP yang telah dikonfigurasi. Alih-alih mengandalkan millis(), kode program dapat dimodifikasi agar berpatokan langsung pada detik absolut internet (misalnya menggunakan kondisi timeinfo.tm_sec % 5 == 0), sehingga seluruh perangkat ESP32 akan mengirimkan paket data suhu dan kelembapan secara serentak dan presisi pada detik yang sama tanpa mengalami pergeseran.